

Acadêmico(a):				RA:
Curso	TECNOLOGIA EM SISTEMAS PARA INTERNET	Período	3º	
Disciplina	HACKATHON			NOTA DA AVALIAÇÃO ESCRITA (0,0 a 6,0)
HACKATHON				
1. A interpretação das questões é parte do processo de avaliação, não sendo permitidas consultas, empréstimos e comunicação entre alunos. 2. Esta avaliação deve ser preenchida à caneta. Respostas à lápis anulam as questões. 3. Rasuras serão consideradas desde que rubricadas na presença do professor ou fiscal. 4. As questões objetivas (marcar X) podem ser assinaladas (à caneta) diretamente nesta folha de avaliação. No entanto, <u>as questões dissertativas devem ser respondidas no papel almaço</u> . Os critérios de correção das questões dissertativas são os seguintes: se citou o conceito básico solicitado; se discorreu sobre o tema de forma organizada; e se efetuou um fechamento da ideia principal. 5. Preencha o cabeçalho do papel almaço mesmo que não o utilize e devolva-o com esta avaliação. 6. Nas avaliações que permitem o uso de calculadoras, não se pode usar a do celular em hipótese alguma. 7. Ao concluir a avaliação, permaneça em seu lugar e comunique ao professor ou fiscal.				
				RUBRICA DO PROFESSOR

PORTAL DE ESTÁGIOS UNIALFA

Conectando Talentos às Oportunidades Locais

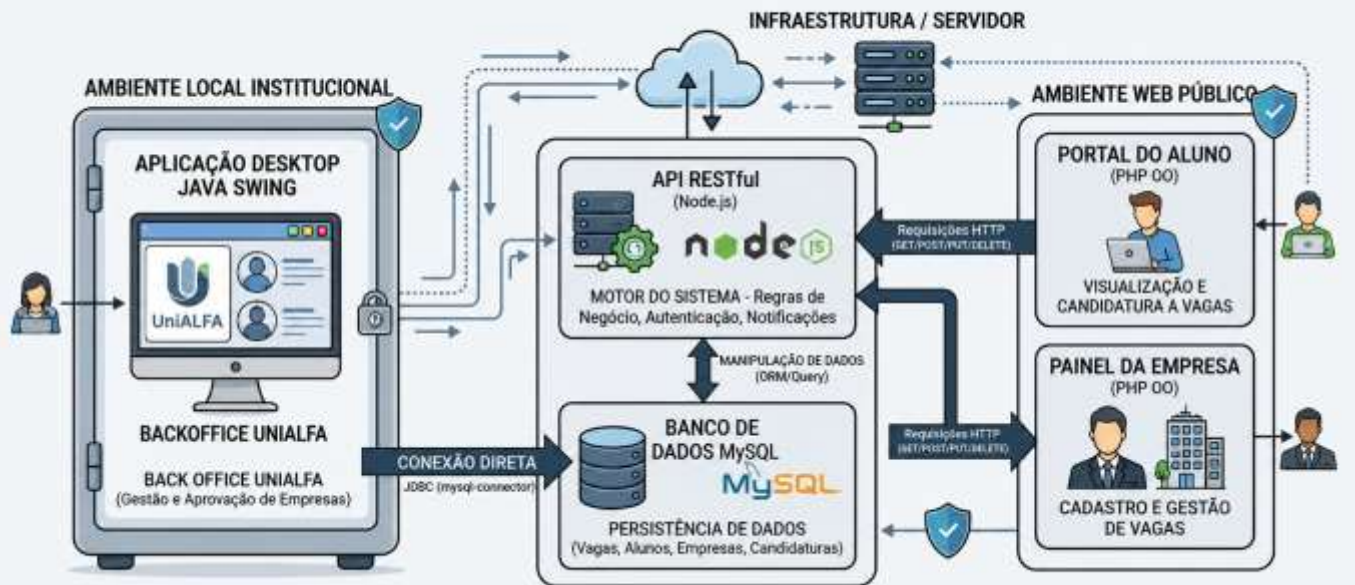
Imagine um ambiente onde possamos criar uma ponte direta entre os alunos da Faculdade UniALFA e as empresas da nossa região que buscam novos talentos. O objetivo é que a proposta seja uma plataforma simples e objetiva, sem processos burocráticos dispersos, na qual o “atrativo principal” — as oportunidades de estágio — sejam visíveis de forma clara e acessível.

Esse é o desafio deste hackathon institucional: criar um **Portal de Estágios** interativo para organizar, promover e gerenciar vagas de estágio e candidaturas. Seu time será responsável por projetar uma solução onde:

- As empresas locais poderão se cadastrar e gerenciar suas vagas de estágio com facilidade através de um painel exclusivo.
- Os alunos poderão visualizar as vagas disponíveis, realizar candidaturas e acompanhar o status de seus processos.
- O sistema possuirá notificações claras sobre o andamento das candidaturas.
- Foco em usabilidade, performance e experiência do usuário são essenciais.

O sistema proposto adota uma arquitetura distribuída, onde cada disciplina do semestre representará uma peça fundamental para o funcionamento do ecossistema do projeto.

PORTAL DE ESTÁGIOS UNIALFA - ARQUITETURA DISTRIBUÍDA DE SISTEMAS



POO Java (Back Office Institucional): Esta é a camada corporativa da solução, voltada para a equipe da própria Faculdade (UniALFA). Através de uma aplicação desktop, a instituição fará a gestão administrativa do portal, como a aprovação de empresas cadastradas e o controle de alunos aptos a estagiar.

Node.js (API RESTful): O motor do sistema. Uma API robusta que centraliza as regras de negócio, fornecendo rotas para vagas, candidaturas e notificações, servindo como ponte de comunicação entre o banco de dados e as interfaces web.

POO PHP (Front-end Web - Alunos e Empresas): A camada pública e de parceiros, onde alunos e empresas acessam a plataforma. O front-end em PHP consome a API Node.JS para renderizar o Portal do Aluno (vagas e candidaturas) e o Painel da Empresa (gestão de suas próprias vagas).

Requisitos por Disciplina

POO (Java Orientado a Objetos)

Para a camada de Back Office Institucional, deve ser construída uma aplicação Java de uso corporativo (que rode em uma infraestrutura local da faculdade).

- **Tecnologias:** A aplicação deve ser construída usando Maven e suas informações devem ser persistidas no Banco de Dados MySQL. Para estabelecer a conexão, adicione a dependência do *mysql-connector* utilizando o Maven.
- **Interface (GUI):** A aplicação contará com uma interface gráfica principal e janelas auxiliares desenvolvidas com a biblioteca Java Swing. A interface deve priorizar clareza, organização e facilidade de utilização pelos colaboradores da instituição.
- **Modelagem de Domínio:** É obrigatória a criação e utilização de classes orientadas a objetos para representar as entidades centrais do sistema, no mínimo: (Aluno, Empresa, Vaga e Candidatura). Outras entidades complementares poderão ser criadas conforme a necessidade da solução proposta.
- **Funcionalidades:** A equipe institucional precisa gerenciar o ecossistema. Cabe aos alunos desenvolver rotinas como: aprovar/bloquear o cadastro de Empresas, gerenciar alunos (ex: importar matriculados) ou gerar relatórios de candidaturas.

- **Funcionalidades** : A aplicação deverá disponibilizar funcionalidades administrativas para gerenciamento do ecossistema do portal, incluindo:
 - Gestão de Empresas:
 - Aprovar empresas cadastradas.
 - Bloquear ou desativar empresas.
 - Consultar informações cadastrais.
 - Gestão de Alunos:
 - Cadastrar, editar e consultar alunos.
 - Importar alunos matriculados através de arquivos texto (.txt).
 - Controlar quais alunos estão aptos a participar dos processos de estágio..
 - Gestão de Vagas e Candidaturas:
 - Consultar vagas cadastradas.
 - Consultar candidaturas realizadas pelos alunos.
 - Visualizar status das candidaturas.
 - Gerar relatórios gerenciais relacionados às vagas e candidaturas..
 - Relatórios (.txt):
 - Empresas cadastradas.
 - Alunos cadastrados.
 - Vagas disponíveis.
 - Candidaturas realizadas e seus respectivos status..
- **Arquitetura e Boas Práticas**: O projeto deverá seguir uma arquitetura organizada e baseada nos princípios da Programação Orientada a Objetos, contendo uma estrutura de pacotes semelhante a (gui, model, service, dao, util). É obrigatório o uso adequado dos conceitos de: Encapsulamento, Herança, Polimorfismo, Abstração, Separação de responsabilidades. O código deverá ser organizado, reutilizável, legível e de fácil manutenção.
- **Requisitos Desejáveis (Diferenciais)**:
 - Sistema de autenticação para acesso ao Back Office Institucional.
 - Controle de perfis de acesso (administrador, coordenador, operador, etc.).
 - Geração de relatórios em formatos adicionais (PDF, CSV).
 - Importação e exportação de dados em formatos complementares.
 - Aplicação de padrões de projeto (Design Patterns) quando apropriado.

POO (PHP Orientado a Objetos)

Para a camada de interação, deve ser construída uma aplicação web em PHP que consuma a API Node.JS. O rigor na modelagem orientada a objetos é essencial.

- **Modelagem de Domínio**: É obrigatória a criação e utilização de classes bem definidas para representar as entidades centrais do negócio, no mínimo: **Aluno**, **Empresa**, **Vaga** e **Candidatura**.
- **Painel da Empresa**: Desenvolver uma área restrita (Back Office) onde a empresa possa realizar o CRUD (Criar, Ler, Atualizar, Excluir) de suas vagas e visualizar a lista de alunos candidatos a cada uma delas.
- **Portal do Aluno**: Desenvolver a interface onde o aluno visualiza a listagem de vagas disponíveis (consumidas da API) e um formulário para submeter sua candidatura.
- **Integração**: A aplicação PHP não deve acessar o banco de dados diretamente; todas as operações de leitura e escrita devem ser feitas através de requisições HTTP à API Node.JS.
- **Boas Práticas**: É fundamental o uso dos conceitos de POO (encapsulamento, herança, polimorfismo, separação de responsabilidades). O código deve ser organizado e limpo. A não aplicação destes fundamentos possui caráter eliminatório.

Node.js

O coração do ecossistema. Vocês deverão desenvolver a API responsável por centralizar toda a lógica de negócio e fornecer os dados para os demais módulos da plataforma.

- **Rotas e Endpoints:** Desenvolver uma API RESTful completa utilizando Node.js, responsável pelo gerenciamento de **Vagas**, **Candidaturas**, e um sistema de **Notificações** (por exemplo, informar quando uma candidatura tiver seu status alterado).
- **CRUD e Persistência:** Implementar todas as operações de criação, consulta, atualização e remoção de dados (CRUD), realizando a persistência das informações em banco de dados relacional.
- **Arquitetura Modular:** A aplicação deverá seguir uma arquitetura organizada e escalável, separando as responsabilidades em camadas distintas como, (Controllers): responsáveis por receber e responder às requisições HTTP, (Services): responsáveis pelas regras de negócio, (Repositories/Modules): responsáveis pelo acesso e manipulação dos dados.
- **Integração com o Front-end PHP:** Toda comunicação entre os sistemas web desenvolvidos em PHP e o banco de dados deverá ocorrer exclusivamente por meio da API Node.js. Os módulos PHP não poderão realizar consultas diretas ao banco de dados, devendo consumir os endpoints disponibilizados pela API através de requisições HTTP. A estrutura do banco de dados deverá ser controlada por meio de *migrations*, garantindo o versionamento das alterações realizadas. Também deverão ser criadas *seeds* para a carga inicial e parametrização dos dados necessários ao funcionamento da aplicação.
- **Padronização das Respostas:** A API deverá retornar respostas em formato JSON, utilizando códigos HTTP apropriados para indicar sucesso, falhas de validação, recursos não encontrados e erros internos do servidor.
- **Segurança e Validação:** As validações de entrada de dados deverão ser implementadas utilizando **Zod**, juntamente com mecanismos de tratamento de erros e validações de negócio que garantam a integridade e consistência das informações processadas pelos endpoints.

DevOps

Durante o Hackathon, as equipes deverão adotar práticas modernas de desenvolvimento e operações (DevOps), demonstrando organização, colaboração e boas práticas de mercado.

Controle de Versão

Todo o desenvolvimento deverá ser realizado utilizando o Git e hospedado no GitHub. O repositório deverá apresentar:

- Histórico consistente de commits, evidenciando a evolução do projeto;
- Utilização adequada de branches para desenvolvimento de funcionalidades;
- Uso de Pull Requests para integração de código e revisão entre membros da equipe;
- Organização e clareza nas mensagens de commit.

Documentação

O projeto deverá conter um arquivo **README.md** completo e atualizado, contemplando:

- Descrição do problema proposto e da solução desenvolvida;
- Objetivos do projeto;
- Tecnologias e ferramentas utilizadas;
- Instruções para instalação e execução local;
- Estrutura do projeto;
- Integrantes da equipe e suas respectivas contribuições;
- Evidências de testes realizados e funcionalidades implementadas.

Colaboração e Organização

As equipes deverão demonstrar trabalho colaborativo por meio do uso adequado das ferramentas de versionamento, registro das atividades desenvolvidas e divisão equilibrada das responsabilidades entre os participantes.

Boas Práticas de Desenvolvimento

Serão considerados na avaliação:

- Organização do código-fonte;
- Padronização e legibilidade do código;
- Modularização da aplicação;
- Tratamento adequado de erros;

User Experience (UX) Design

A primeira impressão e a facilidade de uso determinam o sucesso da plataforma.

- **Guia de estilo e Identidade Visual:** Escolher tipografias e paletas de cores que garantam boa legibilidade e acessibilidade (contraste entre texto e fundo), crie um guia de estilo com a documentação de cores e tipografias escolhidas, defina variáveis para usar em seus protótipos.
- **Protótipos de alta e baixa fidelidade:** Desenvolver um protótipo com um fluxo/jornada de alta fidelidade focando especificamente no **fluxo de candidatura** (desde a tela inicial até a confirmação da candidatura do aluno) e desenvolver protótipos de baixa fidelidade de **todas as telas web**.
- **Experiência:** O design deve ser atrativo, intuitivo e minimizar o atrito (menos cliques para chegar ao objetivo), proporcionando uma experiência visual agradável e funcional tanto para o estudante buscando o primeiro emprego quanto para o RH da empresa.